PHASE 13 · CHAPTER 33

DevSecOps Metrics & Culture

Measurement, Culture & Continuous Improvement

Implement comprehensive DevSecOps metrics, build security-aware culture, enable team self-service, and drive continuous improvement through data-driven decisions and organizational change.

33.1 DORA Metrics

DevOps Research & Assessment — the four key metrics



Deployment Frequency

How often?

5+ /day

Elite teams ship multiple times a day



Lead Time for Changes

How fast?

< 1 hour

Commit to production, end to end



Mean Time to Recovery

How resilient?

< 1 hour

Detect, respond, restore service



Change Failure Rate

How reliable?

< 15%

Elite benchmark is under 5%

Velocity Metrics — Deployment Frequency & Lead Time

Deployment Frequency

Industry Benchmarks

Elite: Multiple deployments per day

High: Weekly deployments

Medium: Monthly deployments

Low: Quarterly or less

Measurement

Tool: GitLab CI/CD logs, ArgoCD

Query: `COUNT(merge_requests.merged) / days`

Improvement Actions

- Automated testing (reduce manual verification)
- Feature flags (deploy without risk)
- Small, incremental changes (easier rollback)
- Blue-green deployments (zero downtime)

Lead Time for Changes

Time from code commit to production

Breakdown (target)

Code review	**< 10 min**
CI pipeline	**< 5 min**
Approval	**< 5 min**
Deployment	**< 10 min**

Total Target: < 30 minutes

Overall target: < 1 hour commit-to-production

Resilience & Reliability — MTTR & Change Failure Rate



Mean Time to Recovery (MTTR)

Time to restore service after failure

Calculation: `incident_resolved_time − incident_detected_time`

Target: < 1 hour

Influencing Factors

- Monitoring (detection speed)
- Alerting (notification speed)
- Runbooks (response procedure)
- Rollback capability (undo bad changes)
- Team familiarity (training)



Change Failure Rate (CFR)

% of changes that cause production incidents

Formula: `(failed_deployments / total_deployments) × 100`

Target: < 15% (elite: < 5%)

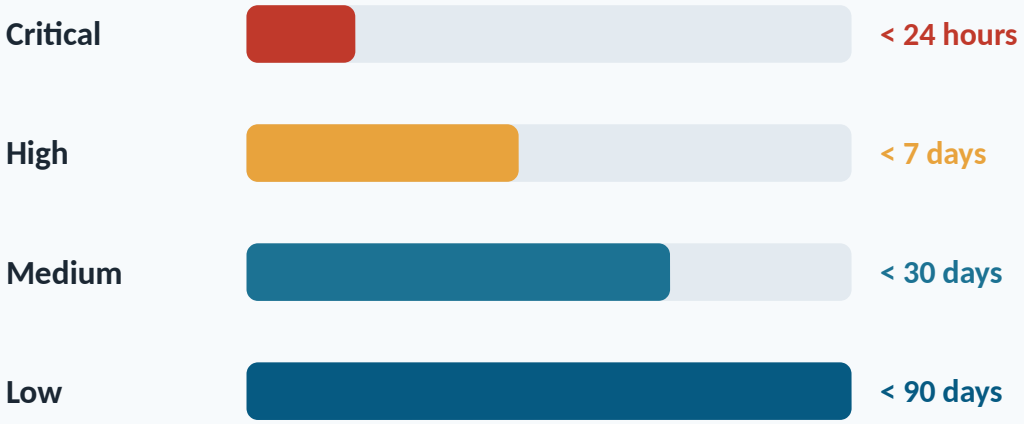
Reduction Strategies

- Automated testing (catch issues early)
- Code review (catch logic issues)
- Staging environment (test before prod)
- Gradual rollouts (catch issues early)
- Monitoring/alerting (detect issues fast)

33.2 Security-Specific Metrics

Vulnerability remediation SLAs by severity

Mean Time to Remediate — by Severity



Root cause on miss: too many vulnerabilities, lack of resources. Actions: temporary mitigation (WAF), isolation, executive escalation.

Vulnerability Density

Vulnerabilities per 1,000 lines of code (KLOC), via SonarQube SAST

< 0.1

target / KLOC

Scanning Coverage

SAST 100% of code · DAST 100% of APIs · Container 100% of images · Dependency & IaC 100%

SLA Compliance

% of vulnerabilities fixed within SLA — e.g. 95/100 critical vulns fixed within 24h

> 95%

target

Application Security Metrics

Code quality and security testing benchmarks

Code Quality

Code Coverage — **> 80%** — JaCoCo, CodeCov

```
lines_executed / total_lines
```

Defect Density — **< 2 / KLOC** — SonarQube

```
defects / lines_of_code
```

Code Review Cycle Time — **< 24 hours** — GitLab/GitHub

```
review_complete − code_submitted
```

Test Execution Time — **< 15 minutes** — CI/CD logs

```
pipeline_end − test_start
```

Security Testing



SAST Findings

0 critical, < 5 high per release — SonarQube



DAST Findings

0 critical per release — OWASP ZAP



Dependency Vulnerabilities

0 critical — Snyk, DependencyCheck



Container Image Vulnerabilities

0 critical per image — Trivy, Grype

33.3 Building Security Culture

Culture Maturity Model — four progressive levels

1

Security Awareness

- Basic training provided
- Incident response playbooks exist
- Security team separate from dev/ops
- Reactive security stance

2

Security Integration

- Security champions in teams
- Security reviews part of PR process
- Some automated security scanning
- Security included in planning

3

Security Ownership

- Developers responsible for security
- Security built into every sprint
- Automated testing on every commit
- Blameless incident post-mortems

4

Security Excellence

- Security automated in pipeline
- Developers write security tests
- Threat modeling standard practice
- Innovation in security practices

33.4 Self-Service Security

Where developer autonomy ends and security-team approval begins



Developers CAN (Self-Service)

- Generate new API keys
- Rotate credentials (scheduled)
- Request certificates
- Deploy to staging
- Run security scans
- Access public documentation
- View security training materials
- Report security concerns
- Access runbooks



Security Team Required

- Deploy to production (requires approval)
- Modify firewall rules
- Change RBAC policies
- Access prod databases directly
- Approve security exemptions
- Modify audit policies
- Request infrastructure changes

33.5 Executive Metrics Dashboard

Monthly DevSecOps health report — all targets met

Velocity

8/day
Deployment Freq.
Target 5+

42 min
Lead Time
Target <1h

18 min
MTTR
Target <1h

8%
Change Failure
Target <15%

Security

0
Critical Vulns
Target 0

2
High Vulns
Target <5

3 days
Vuln MTTR
Target 7d

98%
SLA Compliance
Target >95%

Culture

97%
Training Complete
Target >95%

2%
Phishing Click
Target <5%

8
Security Champions
+2 this mo.

12
Dev Suggestions
Consistent

Reliability & Compliance

99.97%
Uptime
Target 99.95%

2
Incidents
↓ 60%

96%
CIS Benchmark
Target >90%

0
Policy Violations
Target 0

Trend Analysis & Recommendations

What the data says, and where to focus next

Overall

↑ Improving

Security

↑ Stable (proactive)

Velocity

↑ Increasing

Culture

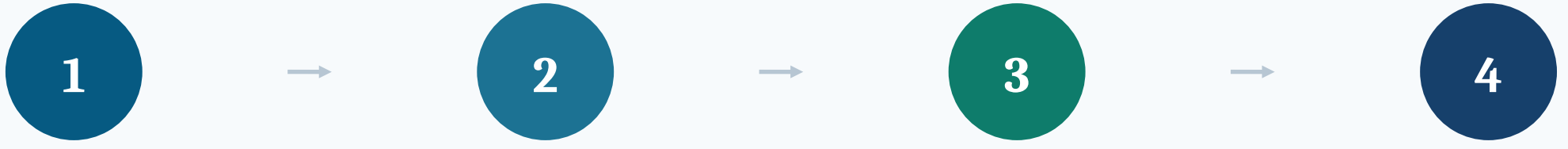
↑ Positive trajectory

Recommendations

- Continue current practices — all metrics on track
- Increase deployment frequency (currently 8/day; industry elite is 10+)
- Expand security champions program — growing engagement
- Schedule advanced security training for architects

33.6 Continuous Improvement Cycle

Plan → Do → Check → Act, applied monthly



PLAN

Review previous month's metrics

DO

Implement improvements

CHECK

Measure results

ACT

Adjust approach; share learnings

Example — April 2024

PLAN: MTTR was 45 min (target <1h) — too many manual steps · **DO:** automated incident response runbooks · **CHECK:** new MTTR = 18 min · **ACT:** rolled out automation to all teams

May 2024 — Measured Improvements

4 → 8/day	**90 → 42 min**	**5 → 0**	**6.5 → 8.2 /10**
Deployment Freq.	Lead Time	Critical Vulns	Team Satisfaction

33.7 Building Team Excellence

Hiring, promotion, and compensation for DevSecOps talent

Junior

0–2 yrs

Learns security fundamentals

Mid

2–5 yrs

Implements security controls

Senior

5+ yrs

Designs security architecture

Junior → Mid

- Automated security testing
- Security vulnerability hunting
- Security documentation
- Team security training delivered

Mid → Senior

- Security architecture design
- Industry presentations / publications
- Security mentoring
- Cross-team security initiatives
- Security innovation

Compensation Premiums

Security expertise +10–20%

Certifications (CISSP, CCSK) +5%

On-call rotation +15%

Innovation bonus +5–10%

33.8 DevSecOps Maturity Progression

A 12-month journey from foundation to excellence



Month 1–3

Foundation

- Basic security controls deployed
- Team trained
- Metrics baseline established
- Processes documented

Month 4–6

Integration

- Security integrated into SDLC
- Automated scanning enabled
- Vulnerability management active
- Incident response tested

Month 7–9

Automation

- Security in every pipeline
- Self-service capabilities
- Advanced threat detection
- Culture transformation

Month 10–12

Excellence

- Industry-leading metrics
- Continuous improvement embedded
- Innovation driving security
- Strategic security partner

Final Takeaway

DevSecOps as a Strategic Advantage

Organizations implementing comprehensive DevSecOps achieve outsized gains vs. low-performers:

46x

more frequent deployments

440x

faster failure recovery

96x

lower change failure rate

- Higher customer satisfaction
- Reduced security incidents
- Met compliance requirements
- Continuous innovation enabled
- Stronger competitive advantage

Chapter 33 Complete · Phase 13: DevSecOps Metrics, Culture & Continuous Improvement